CF Bulk Uploader — Input Format (v1)

This is the contract between authors and the uploader. The parser, validator and writer all implement exactly what is described here — if behaviour and this doc disagree, it's a bug.

Source of truth: `test-data/cf-bulk-upload/cf-import-combined.xlsx` — the only reference workbook. Its layout is the format; this doc spells out the rules behind it. Authors start from the empty sample sheet (**Download sample sheet** on the page), which has the same layout.

File

- Excel workbook, `.xlsx` only (no `.xls`, no `.csv`).
- Limits, set in the OSGi config `ScbCfBulkUploadConfiguration` (defaults: 10 MB, 5,000 rows):
  - a larger file is rejected before it is opened;
  - more than 5,000 rows **with an action** (CREATE / UPDATE / DELETE, all sheets together) is rejected.
    - Rows without an action don't count, so a full report with a few edited rows is fine.
  - Nothing is written when a limit is hit.
- Formulas are allowed — the cell's calculated value is used.

Sheets — one per model

- **The sheet name is the model name** — the model's node name under the conf root

(default `/conf/aemcloudproject/settings/dam/cfm/models`), e.g. `offer-detail`, `merchant-venue`.

- Sheets starting with `_` are ignored, so authors can keep notes in the file. The first sheet of the

sample and of a saved report, `_Guide`, uses this: the authoring rules, the limits and a generated table of every model profile (folder, title and ID columns, required fields).

- A sheet whose name isn't a model with a model profile is reported as an

error and skipped; the other sheets still run.

- **Sheet order and row order don't matter.** One workbook can create venues, merchants that

reference those venues, and offers that reference those merchants. No more `01-venues.xlsx` … `06-offers.xlsx`.

Rows and columns

- **Row 1 is the header.** Data starts on row 2. Completely blank rows are skipped.
- Every header is `action`, `slug` or a **field Property Name from the model** (the name, not the

editor label — `offerStartDate`, not `Offer Start Date`). Case-sensitive. Column order doesn't matter.

- **A header is a single word** — no spaces — because it must match a CF property name.

`Offer Title` is an error. All bad headers in the workbook are reported together.

- Blank header cells are skipped.
- `action` and `slug` are reserved headers (see Action column and

Slug and title); they are not model fields.

- A header that isn't a field of the sheet's model is an error for the whole sheet

(almost always a typo, and silently ignoring it would drop data).

- Every message in the report points at `Sheet › Excel row › column`, using the row number

the author sees in Excel.

Action column

Every sheet has an `action` column (any position) that says what each row does.

`action`	The fragment must…	Otherwise
`CREATE`	**not exist** yet. Required fields must be filled.	Error: already exists — nothing is overwritten.
`UPDATE`	**already exist**. Every column in the sheet is applied; a blank cell clears its field.	Error: not found — catches typos in the slug.
`DELETE`	**already exist**, and nothing may point to it. Only `action` and `slug` are read.	Not found: error. Referenced or published: `WARN`, not deleted — see Deleting.

- The sample and saved reports have a dropdown with the three values on the action column (rows 2

to 5,001). In a workbook built by hand any case works: `create`, `Create`, `CREATE` are the same.

- A row with a blank `action` is skipped: it is not checked, written or listed in the results.

Its ID still counts when new IDs are generated (see Generated IDs).

- If **no row** in a sheet has an action, the sheet gets one `SKIP` line in the results

("Sheet: no row has an action") and the other sheets carry on. `SKIP` is not a failure.

- Any other value (e.g. `REMOVE`) is an error for that row, and the whole workbook is rejected.

Deleting

- `DELETE` rows run **after** every `CREATE` and `UPDATE` row of the workbook, sheet by sheet in

reverse profile order (offers first, venues last). So an offer and the CTA it points to can be deleted in one run, and an `UPDATE` that removes a reference runs before the delete it unblocks.

- A fragment is **not deleted** (result `WARN`, with the reason) when:
  - another fragment under the picked folder stores its path, or a page or experience fragment
    - references it (AEM's reference search) — the first five are listed;
  - it is still published (last replication action is Activate).
- A fragment deleted earlier in the same run, dry or final, no longer counts as a reference.
- `WARN` is not a failure: it doesn't block the final run. The fragment stays until the references

are removed or it is unpublished, and the row can be run again.

- In a dry run, an `UPDATE` that would remove a reference is not written, so the delete it unblocks

still shows `WARN`; the final run deletes it.

Model profiles

The uploader is generic; what's specific to a model lives in OSGi config, not in the spreadsheet: one **SCB CF Model Profile** factory configuration per model, in `ui.config/.../osgiconfig/config.author/com.aemcloudproject.core.services.impl.ScbModelProfileImpl~<model>.cfg.json`. A sheet whose name matches no profile's `model` fails with "not a known CF model". To support a new model, add one such file — no code change. A profile says, per model:

Setting	Meaning
`model`	Model node name under the models path — also the sheet name.
`folder`	Folder for this model's fragments, relative to the **fragment folder picked on the page** (pre-filled from `fragmentRootPath` in `ScbCfBulkUploadConfiguration`, default `/content/dam/aemcloudproject/cfs/offer-listing`). Created as `sling:OrderedFolder` if missing.
`titleField`	Field whose value becomes the fragment's title.
`idField`	Numeric ID field filled automatically on `CREATE` (optional).
`startId`	First ID used when neither the sheet nor DAM has an ID yet.
`sortField`	Field a saved report sorts the model's sheet by, latest or highest first (optional; default: `idField`, smallest first).
`order`	Position of the model's sheet in the sample and report, lower first; deletes run in reverse order.

Profiles for this project:

Sheet / model	folder	titleField	idField	startId	sortField	order
`merchant-venue`	`merchants/venues`	`name`	—	—	—	1
`merchant-details`	`merchants`	`merchantName`	`merchantID`	100001	—	2
`category`	`categories`	`name`	`ID`	1001	—	3
`offer-card`	`cards`	`name`	`ID`	1001	—	4
`offer-cta`	`offer-cta`	`label`	—	—	—	5
`offer-detail`	`offers`	`offerTitle`	`offerID`	100001	`offerEndDate`	6

Slug and title

- **Slug** = the `slug` column: the fragment's node name. It must already be a valid name —

lowercase letters, digits, `-`, `_` — otherwise the row is an error. It is never auto-slugified, so what the author types is exactly the path they get.

- The slug is **only used for the path**; it is not stored in the fragment, and the models have no

slug field.

- **Fragment path** = picked folder + profile `folder` + slug. `CREATE` requires that path to

be free, `UPDATE` and `DELETE` require a fragment there. A different slug is a different fragment: a fragment can't be renamed through the upload.

- **Title** = the `titleField` value; if that cell is empty on `CREATE`, the slug is used.

On `UPDATE`, an empty title cell keeps the current title.

- Not checked yet: the same slug twice in one sheet. In a final run the second `CREATE` fails with

"already exists"; the dry run doesn't catch it (open decision).

Generated IDs

- A `CREATE` row whose `idField` cell is empty gets an ID; a typed ID is kept as is.

`UPDATE` rows never get a new ID (an empty ID cell keeps the stored one).

- One counter per model. The first new ID is **the highest ID + 1**, counting every row of the sheet

with a numeric ID (including rows with a blank `action`) **and** every existing fragment of the model in its folder in DAM. If there is none, `startId` is used.

- Further new rows get +1 each, top to bottom. A row that fails does not use up an ID.
- The ID is shown in the results, e.g. `…/offers/summer-sale (offerID 100004)`, in a dry run too.
- Because DAM is counted, an empty sample sheet continues the numbering correctly.

Cell values by field type

The type of each field is read from the model, so headers carry no type hints.

Model field type	Cell value
Single-line text	Text as-is. Leading/trailing whitespace trimmed.
Multi-line text	Text as-is, line breaks preserved. Written with the field's default content type — all multi-line fields in this project are `text/html`, so put HTML in the cell.
Number (integer)	Numeric cell, or text like `10001`. `3.5` is an error.
Number (fraction)	Numeric cell, or text like `1.2935`.
Boolean	Excel `TRUE` / `FALSE`, `1` / `0`, or text `true/false`, `yes/no` (case-insensitive).
Date and time	ISO-8601 text — `2026-10-01T00:00:00.000+05:30`, `2026-10-01T09:30`, `2026-10-01` — or an Excel date cell. Values without an offset use the configured time zone (OSGi, default `Asia/Kolkata`).
Enumeration	Option **value** (not its label).
Tags	Tag IDs, e.g. `aemcloudproject:offers/dining`.
Content reference	Absolute repository path (`/content/dam/…`, `/content/aemcloudproject/…`) **or** an external `http(s)://` URL, stored as-is.
Fragment reference	Absolute path, e.g. `/content/dam/aemcloudproject/cfs/offer-listing/cards/gold-card` — see below.
JSON object	Raw JSON text. Must parse.

Multi-value fields

Multi-value fields (all of them are fragment references in this project): **one path per line** inside the cell (Alt+Enter in Excel, Option+Return on Mac).

```
/content/dam/aemcloudproject/cfs/offer-listing/cards/platinum-card
/content/dam/aemcloudproject/cfs/offer-listing/cards/gold-card
```

How the uploader recognises them: a cell whose text **starts with** `/content` is split on line breaks; each line is trimmed and blank lines are dropped. Two or more lines become a list of values, one line stays a

single value. Every other cell is kept as-is — so multi-line HTML such as `address` or `offerDescription` (which starts with `<p>`) is never split.

> Pending: a future model with a multi-value field whose values don't start with `/content` > (e.g. multi-value text or tags) would not be split. Revisit when headers are checked > against the model.

Fragment references

Each value in a fragment-reference cell is an **absolute path**, e.g. `/content/dam/aemcloudproject/cfs/offer-listing/cards/gold-card`. Bare names (`gold-card`) are not supported. The target must exist in DAM **or be created by a row in the same workbook**, and must use the model the field allows.

Self-references (`category.parent`, `offer-card.parent`) work the same way — a row may point at a row below it.

> Not built yet: the writer stores reference paths as given and does not check that the target > exists. The two-pass write and the checks below are the planned behaviour.

Missing referenced content (planned)

Reference	Target missing →
Fragment reference	**error** — row not written. A row that references an *errored* row is also an error, with the reason.
Content reference (path)	**warning** — row written, flagged in the report (assets are often uploaded later).
Content reference (URL)	not checked.

Empty cells vs. clearing a field

For `UPDATE`, start from a saved report (**Save report to DAM**): the sheet then holds every current value, and a blank cell means "this field has no value". Or keep only the columns you change.

Cell	`CREATE`	`UPDATE`
value	field set	field overwritten
blank	field empty	field **cleared**
blank ID field (`idField`)	ID generated	ID **kept**
column not in the sheet	field empty	field **not touched**
`#CLEAR`	row fails: `#CLEAR is no longer used — leave the cell empty to clear the field`	same

- Only columns present in the sheet can clear fields, so a sheet with just `action`, `slug` and

one more column changes that one column and nothing else.

- The fragment's own title (`jcr:title`) is only changed by a non-blank title cell; a blank title

cell clears the title field but keeps the title shown in Assets.

- Results list the fields a row clears that had a value, in dry runs too:

`…/offer-cta/book-now (cleared: deeplink)`. Clearing an already-empty field is not reported.

Required fields

The CF model is the only source: a field is mandatory when *Required* is ticked for it in the model editor (read from the model's dialog definition). Nothing is configured in OSGi.

- `CREATE`: every required field must have a value, otherwise the row fails with

`offerTitle: required` (all missing fields in one message). A missing column counts as empty. An empty ID field that gets generated counts as filled.

- `UPDATE`: a required column that is in the sheet may not be blank (it would clear the field):

the row fails with `offerTitle: required`. A required column left out of the sheet is fine, and a blank ID field keeps the current ID.

- The `slug` column must be present in the sheet and filled on every row with an action.
- The sample and report show `slug` and the required fields as red bold headers, from the same model flags.

Behaviour summary

- **Explicit actions:** each row is `CREATE`, `UPDATE` or `DELETE` (see Action column).

Updating or deleting a fragment whose existing model differs from the sheet's model is an error (never silently re-modelled).

- Rows are independent: one bad row is reported and skipped; it doesn't fail the file.
- **Dry run** runs full validation and reports what *would* be created, updated, deleted or failed,

without writing anything.

- **Final run** writes. It is enabled only after a dry run of the same workbook and folder with no

`FAIL` rows, and the server refuses it when the workbook changed in DAM after that dry run.

- Each final run saves a results report, `cf-upload-run-<yyyy-MM-dd-HH-mm-ss>.xlsx`, in the report

folder (`reportFolder`, default `/content/dam/aemcloudproject/reports`).

- **Save report to DAM** saves every fragment under the picked folder as

`offer-listing-report-<yyyy-MM-dd-HH-mm-ss>.xlsx` in the same folder; each save is a new file.

- The run happens in the request; the page shows a spinner until the results come back.

Out of scope for v1

Variations, per-cell content-type override, asset upload from the file, publishing and unpublishing, renaming fragments, auto-creating stub fragments for unknown references. All can be added later without breaking this format.

Model field reference (this project)

`*` = required on create. Arrows show the model a fragment reference allows. Field order = model order = sample and report column order, after `action` and `slug`. The models have no slug field.

`merchant-venue`

Property name	Field label	Type
`name`	Name	Single-line text
`address`	Address	Multi-line text (HTML)
`latitude`	Latitude	Single-line text
`longitude`	Longitude	Single-line text
`telephone`	Telephone	Single-line text
`ophr`	Ophr	Single-line text
`fax`	Fax	Single-line text

`merchant-details`

Property name	Field label	Type
`merchantID`	Merchant ID	Single-line text `*` (digits; generated on CREATE)
`merchantName`	Merchant Name	Single-line text
`merchantLogo`	Merchant Logo	Boolean
`merchantVenues`	Merchant Venues	Fragment reference, multiple → `merchant-venue`

`category`

Property name	Field label	Type
`ID`	ID	Number (integer) `*`
`name`	Name	Single-line text
`categoryIconPath`	Category Icon Path	Content reference
`parent`	Parent	Fragment reference → `category`

`offer-card`

Property name	Field label	Type
`ID`	ID	Number (integer)
`name`	Name	Single-line text
`logo`	Logo	Content reference
`parent`	Parent	Fragment reference → `offer-card`

`offer-cta`

Property name	Field label	Type
`label`	Label	Single-line text
`url`	URL	Content reference (external URLs allowed)
`deeplink`	Deep Link	Content reference

`offer-detail`

Property name	Field label	Type
`offerID`	Offer ID	Number (integer) `*`
`offerTitle`	Offer Title	Single-line text `*`
`offerSummary`	Offer Summary	Multi-line text (HTML)
`offerDescription`	Offer Description	Multi-line text (HTML)
`offerImage`	Offer Image	Content reference
`alternateText`	Alternate text	Single-line text
`hotPromo`	Hot Promo	Boolean
`offerStartDate`	Offer start date	Date and time
`offerEndDate`	Offer end date	Date and time
`offerURL`	Offer URL	Content reference
`offerLabel`	Offer Label	Single-line text
`offerEmail`	Offer Email	Single-line text
`offerTnc`	Offer TNC	Multi-line text (HTML)
`offerType`	Offer Type	Single-line text
`offerPromoCodes`	Offer Promo Codes	Single-line text
`offerBadge`	Offer Badge	Single-line text
`offerTags`	Offer Tags	Single-line text
`offerLocation`	offer Location	Single-line text
`offerLocationType`	Offer Location Type	Single-line text
`offerCategories`	Offer Categories	Fragment reference, multiple → `category`
`offerMerchants`	Offer Merchants	Fragment reference, multiple → `merchant-details`
`offerCards`	Offer Cards	Fragment reference, multiple → `offer-card`
`offerCta`	Offer CTA	Fragment reference, **single** → `offer-cta`
`offerOrigin`	Offer Origin	Single-line text
`brCode`	Br code	Single-line text
`qrCode`	Qr code	Single-line text
`preview`	Preview	Boolean
`cache`	Cache	Boolean